<TÊN TRƯỜNG / KHOA>

HỌC PHẦN HỌC MÁY CƠ BẢN

BÁO CÁO PROJECT KẾT THÚC HỌC PHẦN

PROJECT 13

DỰ ĐOÁN KHẢ NĂNG HỦY ĐẶT PHÒNG TẠI THỜI ĐIỂM ĐẶT

Chủ đề: Bài 5 — Hồi quy logistic: phân loại bằng xác suất

Lớp	12523W.1
Giảng viên	PGS.TS. Nguyễn Văn Hậu
Thành viên 1	<Họ tên – MSSV>
Thành viên 2	<Họ tên – MSSV>
Dữ liệu	Hotel Booking Demand (CC BY 4.0)
Mã nguồn	github.com/Nguyenvan01/hotel-booking-cancellation-prediction

Tháng 10/2026

MỤC LỤC

1. Tóm tắt....4
2. Bối cảnh và câu hỏi nghiên cứu....4
2.1 Bối cảnh thực tiễn....4
2.2 Câu hỏi nghiên cứu và đặc tả bài toán....5
2.3 Đóng góp của nhóm....5
3. Dữ liệu và giấy phép....5
3.1 Nguồn và quyền sử dụng....5
3.2 Chất lượng dữ liệu....6
3.3 Dòng trùng lặp và booking theo đoàn....6
3.4 Khám phá dữ liệu trên tập train....6
4. Thời điểm dự đoán và kiểm soát rò rỉ....8
4.1 Thời điểm dự đoán quyết định tập đặc trưng....8
4.2 Bằng chứng rò rỉ từng biến....8
4.3 Tập đặc trưng cuối cùng....9
4.4 Các biện pháp chống rò rỉ trong quy trình....9
5. Phương pháp....9
5.1 Hồi quy logistic....9
5.2 Pipeline tiền xử lý....10
6. Thiết kế thí nghiệm....10
6.1 Chia dữ liệu theo thời gian....10
6.2 Baseline....10
6.3 Chọn tham số trên validation....11
6.4 Ablation....11
6.5 Khoảng biến thiên và giao thức test....11
6.6 Chọn ba ngưỡng hành động....11
7. Kết quả....11
7.1 Thí nghiệm 1: baseline và mô hình đầy đủ....11
7.2 Chọn tham số và ablation....12
7.3 Thí nghiệm 2: calibration....13
7.4 Thí nghiệm 3: ba ngưỡng hành động....14
7.5 Thí nghiệm 4: hệ số và odds ratio....14
8. Phân tích lỗi....16
8.1 Theo nhóm....16
8.2 Theo thời gian đến....17
8.3 Các trường hợp FP/FN điển hình....17
8.4 Trường hợp ngoài miền....18
9. Ứng dụng web và API....18
9.1 Kiến trúc....18
9.2 Kiểm tra đầu vào....19
9.3 Giao diện....20
9.4 Kiểm thử....21
10. Đạo đức và giới hạn....22
10.1 Sử dụng có trách nhiệm....22
10.2 Công bằng và quyền riêng tư....22
10.3 Giới hạn....22

11. Kết luận.....23
12. Tài liệu tham khảo.....24
Phụ lục A. Hướng dẫn tái lập.....25
Phụ lục B. Bảng theo dõi thí nghiệm (validation).....26
Phụ lục C. Từ điển dữ liệu.....27
Phụ lục D. Chuẩn bị vấn đáp.....28
Phụ lục E. Kịch bản demo (5–7 phút).....29
Phụ lục F. Phân công, nhật ký và khai báo công cụ AI.....30

1. Tóm tắt

Khách sạn cần biết sớm booking nào có rủi ro bị hủy để chủ động gọi xác nhận và lập kế hoạch công suất phòng, nhưng không được gây bất lợi cho khách. Báo cáo này trả lời câu hỏi: **với thông tin có ngay lúc booking được tạo, hồi quy logistic dự đoán xác suất hủy và hỗ trợ chọn ngưỡng hành động ra sao?**

Nhóm dùng bộ dữ liệu *Hotel Booking Demand* (Antonio, de Almeida & Nunes, 2019; CC BY 4.0) gồm 119.390 booking của một khách sạn thành phố và một resort ở Bồ Đào Nha. Dữ liệu được tải bằng script có kiểm tra SHA-256, chia theo **ngày đến** thành train (07/2015–12/2016), validation (01–04/2017) và test (05–08/2017). Ngoài 5 biến đề bài yêu cầu loại, nhóm loại thêm 7 biến nghi được ghi nhận sau thời điểm đặt hoặc có rủi ro công bằng (ví dụ chỗ đậu xe: 100% booking có chỗ đậu xe đều không hủy; quốc tịch). Mô hình chính là `LogisticRegression` trong một `Pipeline` (one-hot, log1p + chuẩn hóa) với 16 đặc trưng; tham số `C`, penalty và `class_weight` được chọn theo log loss trên validation.

Trên tập test chỉ dùng một lần, mô hình đạt **ROC-AUC 0,727** (khoảng tin cậy 95%: 0,709–0,748), PR-AUC 0,674, log loss 0,583, vượt cả hai baseline (logistic chỉ với lead time: ROC-AUC 0,619; tỷ lệ hủy chung: log loss 0,679). Xác suất được hiệu chỉnh tốt ở mức tổng thể (trung bình dự đoán 39,5% so với tỷ lệ thực tế 40,6%). Ba ngưỡng hành động chọn trên validation cho phép đánh đổi giữa bắt được 91% booking hủy (gắn cờ 76% booking) và precision 0,70 khi chỉ gắn cờ khoảng 20%. Hiệu năng giảm so với validation (ROC-AUC 0,783) do trôi theo thời gian, yếu hơn ở nhóm Online TA và dự đoán thấp cho Resort Hotel. Mô hình được đóng gói thành ứng dụng web (FastAPI + React) có kiểm tra đầu vào, cảnh báo ngoài miền, dashboard đánh giá và model card.

Từ khóa: hồi quy logistic, hủy đặt phòng, rò rỉ dữ liệu, calibration, chọn ngưỡng, odds ratio.

2. Bối cảnh và câu hỏi nghiên cứu

2.1 Bối cảnh thực tiễn

Trong ngành khách sạn, hủy đặt phòng gây ra phòng trống không bán lại kịp và làm sai lệch kế hoạch nhân sự, thực phẩm và giá. Đặt phòng trực tuyến giúp khách dễ đặt nhưng cũng dễ hủy: trong dữ liệu dùng ở đây, khoảng 37% booking bị hủy. Nếu ước lượng được rủi ro hủy **ngay lúc nhận booking**, bộ phận lễ tân có thể ưu tiên gọi xác nhận những booking rủi ro cao trong giới hạn nhân lực.

Yêu cầu quan trọng của đề bài là **không gây bất lợi cho khách**: mô hình chỉ dùng để ưu tiên một cuộc gọi xác nhận, không dùng để tự động hủy, tăng giá hay phân biệt đối xử.

2.2 Câu hỏi nghiên cứu và đặc tả bài toán

Câu hỏi: với thông tin có ngay lúc đặt, logistic regression dự đoán xác suất hủy và hỗ trợ chọn ngưỡng hành động ra sao? Đặc tả được cố định trong `config.yaml` và mã nguồn trước lần chạy test đầu tiên (Bảng 1).

Bảng 1. *Đặc tả bài toán học máy* Nguồn: docs/project_brief.md.

Thành phần	Nội dung
Loại bài toán	Phân loại nhị phân theo xác suất; nhãn `is_canceled` (1 = hủy hoặc no-show)
Đơn vị quan sát	Một booking tại thời điểm được tạo
Thời điểm dự đoán	Ngay khi booking được nhập vào hệ thống (lead time ngày trước ngày đến)
Đầu vào	16 biến có sẵn lúc đặt (mục 4.3)
Đầu ra	Xác suất hủy và nhóm ưu tiên xác nhận (Rất cao / Cao / Trung bình / Thấp)
Chỉ số chính	ROC-AUC, PR-AUC, log loss, Brier, precision/recall tại ngưỡng, calibration curve
Phạm vi	Gộp 2 khách sạn (City Hotel và Resort Hotel), `hotel` là một đặc trưng
Tiêu chí thành công	Chạy lại được trên máy khác; vượt baseline hoặc giải thích rõ; chứng minh không rò rỉ

2.3 Đóng góp của nhóm

- Pipeline tái lập được từ dữ liệu thô đến ứng dụng bằng một lệnh (`make all`), có kiểm tra checksum, 51 test tự động và kiểm thử end-to-end trên trình duyệt.
- Phân tích rò rỉ theo từng biến có bằng chứng định lượng, kèm một thí nghiệm minh họa rò rỉ làm ROC-AUC tăng giả tạo.
- Đủ bốn thí nghiệm bắt buộc (baseline, calibration, ba ngưỡng, odds ratio) với tách bạch validation/test và khoảng tin cậy bootstrap theo cụm ngày đến.
- Phân tích lỗi theo nhóm và theo thời gian, ứng dụng web có giải thích trực quan đóng góp của từng biến.

3. Dữ liệu và giấy phép

3.1 Nguồn và quyền sử dụng

Dữ liệu là *Hotel Booking Demand Datasets* do Antonio, de Almeida và Nunes công bố trên *Data in Brief* (2019, tập 22, tr. 41–49, doi:10.1016/j.dib.2018.11.126) theo giấy phép **CC BY 4.0**, cho phép dùng cho học tập khi trích dẫn nguồn. Tệp `mmc2.zip` (phần supplementary của bài báo) chứa `H1.csv` (Resort Hotel, 40.060 dòng) và `H2.csv` (City Hotel, 79.330 dòng), tổng **119.390 booking**, 31 cột, ngày đến từ 01/07/2015 đến 31/08/2017. Dữ liệu đã được tác giả ẩn danh hóa.

Script `src/data.py` tải tệp zip, kiểm tra SHA-256 của tệp zip và từng CSV với giá trị ghi trong `config.yaml`, kiểm tra số dòng, tên và kiểu của 32 cột (31 cột gốc và cột `hotel`). Dữ liệu thô nằm trong `data/raw/` và **không được commit** lên Git.

`data/README.md` ghi nguồn, ngày tải, tệp sử dụng, checksum, giấy phép và trích dẫn; `data/data_dictionary.csv` ghi kiểu, đơn vị, vai trò và **thời điểm biến có sẵn** của cả 33 biến (Phụ lục C).

3.2 Chất lượng dữ liệu

Bảng 2. *Tóm tắt kiểm tra chất lượng dữ liệu thô* Nguồn: reports/data_quality.md (sinh bởi src/data.py).

Kiểm tra	Kết quả	Xử lý
Số dòng / cột	119.390 / 32	Khớp kỳ vọng
`children` thiếu	4 dòng	Loại dòng (không tự điền)
`country` thiếu	488 dòng (0,41%)	Biến bị loại khỏi đặc trưng
`agent` / `company` = NULL	16.340 (13,7%) / 112.593 (94,3%)	NULL nghĩa là không qua đại lý/công ty; hai biến bị loại
Dòng trùng hoàn toàn	31.994 (26,8%)	Giữ (booking theo đoàn), xem mục 3.3
Tổng số khách = 0	180 dòng	Loại (booking không hợp lệ)
`adr` < 0 / max	1 dòng / 5.400	Ghi nhận; `adr` không dùng trong mô hình chính
Hạng mục `Undefined`	meal 1.169; market_segment 2; distribution_channel 5	Giữ nguyên là hạng mục riêng

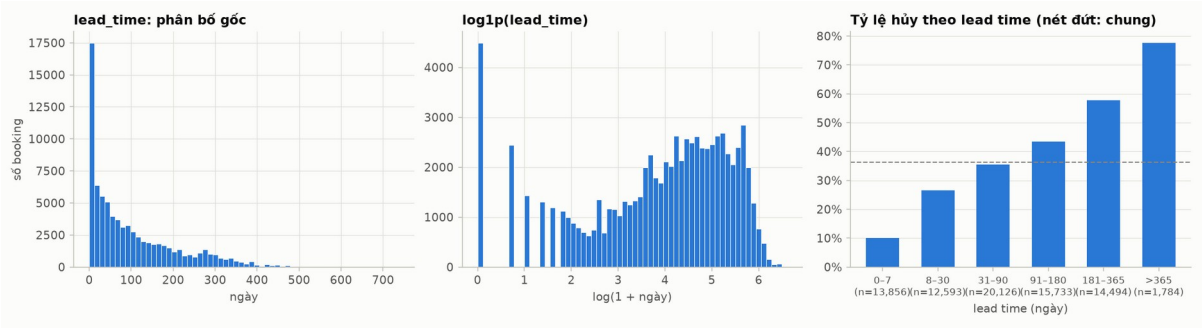
Sau làm sạch còn **119.206** booking (loại 184 dòng, mỗi dòng có lý do ghi trong `reports/data_quality.md`). Tỷ lệ hủy là 41,8% ở City Hotel và 27,8% ở Resort Hotel; toàn bộ là 37,1%, tức mất cân bằng nhẹ. Vì vậy nhóm không dùng accuracy làm thước đo chính.

3.3 Dòng trùng lặp và booking theo đoàn

Dữ liệu không có mã booking. Trên train, 45,7% dòng thuộc các nhóm giống hệt nhau về đặc trưng, ngày đến và nhãn; nhóm lớn nhất có 180 booking. Các dòng trùng tập trung ở phân khúc *Groups* (36,6%) và *Offline TA/TO* (32,0%), phù hợp với nhiều phòng của cùng một đoàn. Nhóm giữ lại các dòng này vì không phân biệt được lỗi nhân đôi với phòng thật. Hai biện pháp đảm bảo đánh giá trung thực: (1) chia theo ngày đến nên các bản trùng luôn nằm cùng một tập; (2) khoảng tin cậy dùng bootstrap theo cụm ngày đến (mục 6.5).

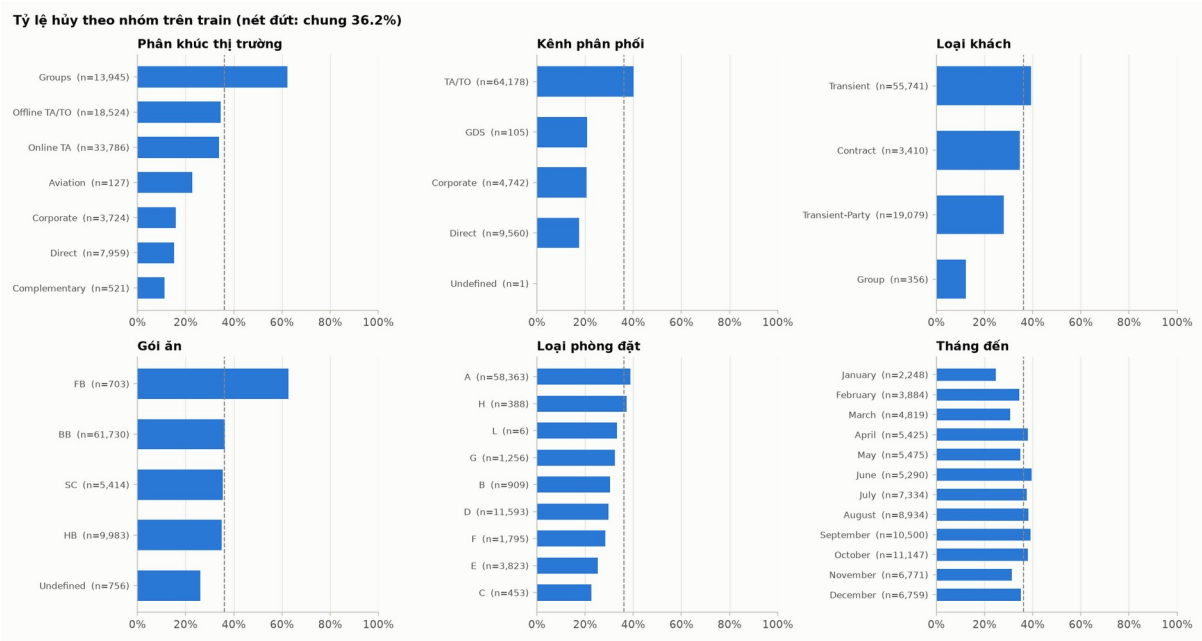
3.4 Khám phá dữ liệu trên tập train

Toàn bộ EDA (`notebooks/eda.ipynb`) chỉ đọc tập train. Mỗi biểu đồ dẫn tới một quyết định tiền xử lý.



Hình 1. *Phân bố lead time (gốc và log1p) và tỷ lệ hủy theo khoảng lead time trên train.* Nguồn: notebooks/eda.ipynb, tập train.

`lead_time` lệch phải mạnh (trung vị 65 ngày, tối đa 737 ngày). Tỷ lệ hủy tăng từ 10,0% (0–7 ngày) lên 26,5% (8–30), 35,6% (31–90), 43,5% (91–180), 57,7% (181–365) và 77,6% (trên 365 ngày): tăng nhanh ở đầu rồi chậm dần, gần với dạng log. Các biến đếm khác (số đêm, số người, số booking trước) cũng lệch phải (độ lệch 1,3 đến 27). **Quyết định:** dùng `log1p` rồi chuẩn hóa cho các biến đếm (D-12); kiểm chứng bằng ablation A1.



Hình 2. *Tỷ lệ hủy theo các biến phân loại trên train (nét đứt: tỷ lệ chung).* Nguồn: notebooks/eda.ipynb, tập train.

Phân khúc *Groups* và kênh *TA/TO* hủy nhiều, trong khi *Direct*, *Corporate*, *Complementary* hủy ít; loại khách *Transient* hủy nhiều hơn *Contract*. Tỷ lệ hủy dao động theo tháng đến. Một số hạng mục rất hiếm (phòng `L`: 6 dòng). **Quyết định:** one-hot cho 7 biến phân loại, hạng mục dưới 50 dòng gộp thành nhóm hiếm; giữ tháng đến, không dùng năm/tuần/ngày (D-09, D-12).

Về lịch sử khách, `previous_cancellations = 1` có tỷ lệ hủy 97,0% (5.854 booking) trong khi giá trị 2 chỉ 41,6%. Quan hệ không đơn điệu này là dấu hiệu của các đợt

booking theo đoàn bị hủy hàng loạt. Biến vẫn được giữ vì đề bài liệt kê rõ "lịch sử hủy có trước" là đầu vào, nhưng rủi ro được nêu trong model card (D-08).

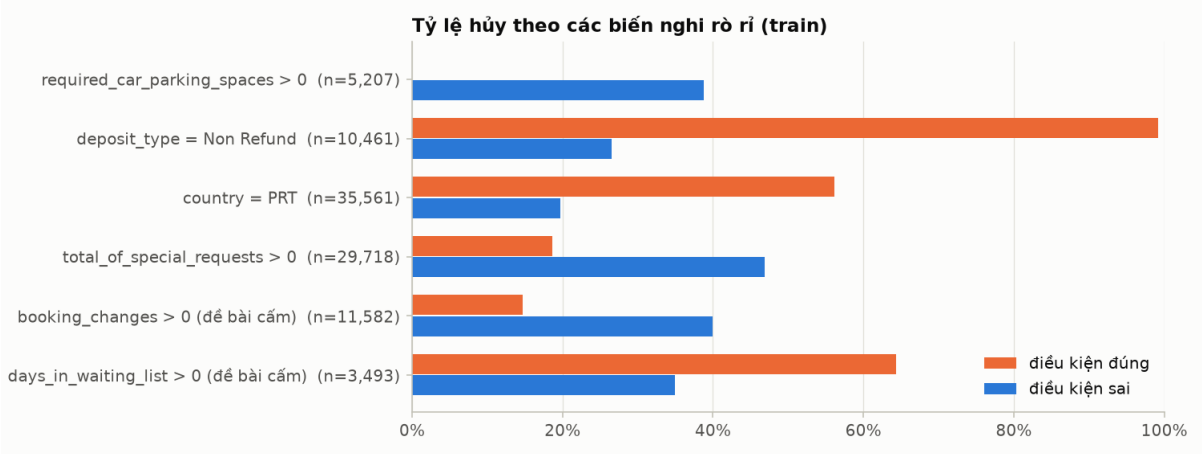
4. Thời điểm dự đoán và kiểm soát rò rỉ

4.1 Thời điểm dự đoán quyết định tập đặc trưng

Mô hình chạy **ngay khi booking được tạo**. Một biến chỉ được dùng nếu giá trị của nó chắc chắn đã có tại thời điểm đó và không bị cập nhật về sau. Nếu dùng biến phát sinh sau lúc đặt, mô hình sẽ đạt điểm cao trên dữ liệu lịch sử nhưng không thể chạy khi triển khai, vì lúc đó biến chưa tồn tại. Đây chính là **rò rỉ dữ liệu** (data leakage).

4.2 Bằng chứng rò rỉ từng biến

Ngoài 5 biến đề bài yêu cầu loại, nhóm xét từng biến còn nghi vấn. Dấu hiệu chính là một biến "biết lúc đặt" lại tách lớp gần như tuyệt đối hoặc ngược với logic kinh doanh (Hình 4).



Hình 3. *Tỷ lệ hủy khi điều kiện đúng (cam) và sai (xanh) với các biến nghi rò rỉ, trên train.* Nguồn: notebooks/eda.ipynb, tập train.

Bảng 3. *Quyết định giữ/loại các biến nghi vấn* Nguồn: docs/decisions.md, data/data_dictionary.csv.

Biến	Bằng chứng (train)	Quyết định
`reservation_status`, `reservation_status_date`, `assigned_room_type`, `booking_changes`, `days_in_waiting_list`	Phát sinh sau lúc đặt (theo định nghĩa)	Loại (đề bài)
`required_car_parking_spaces`	5.207 booking có chỗ đậu xe: 0% hủy	Loại (D-03)
`country`	PRT hủy 56,2%, nước khác 19,7%; nghi sửa lúc check-in; rủi ro phân biệt quốc tịch	Loại (D-04)
`deposit_type`	Non Refund: 99,2% hủy, ngược logic kinh doanh	Chỉ dùng ablation (D-05)
`adr`, `total_of_special_requests`	Tính từ giao dịch / có thể bổ sung sau lúc đặt	Chỉ dùng ablation (D-05)
`agent`, `company`	ID nhiều giá trị, có thể điền sau (thanh toán)	Loại (D-06)
Số đêm, số khách	Có thể bị sửa nhưng không có dấu hiệu tách lớp bất thường	Giữ, nêu rủi ro (D-07)

4.3 Tập đặc trưng cuối cùng

Mô hình chính dùng **16 đặc trưng**: 7 phân loại (`hotel`, `arrival_date_month`, `meal`, `market_segment`, `distribution_channel`, `reserved_room_type`, `customer_type`), 8 biến đếm (`lead_time`, số đêm cuối tuần và trong tuần, `adults`, `children`, `babies`, `previous_cancellations`, `previous_bookings_not_canceled`) và 1 biến nhị phân (`is_repeated_guest`).

4.4 Các biện pháp chống rò rỉ trong quy trình

- Chia train/validation/test theo ngày đến **trước** mọi bước học; encoder, scaler đều nằm trong `Pipeline` và chỉ fit trên train.
- Test tự động `tests/test_leakage.py` khẳng định không cột bị cấm nào nằm trong danh sách đặc trưng, data dictionary khớp với mã nguồn, pipeline đã lưu chỉ nhận đúng 16 cột, và `src/train.py` không đọc tập test.
- Mô hình, tham số và ngưỡng được commit trước khi chạy test; mỗi lần chạy test được ghi vào `reports/test_runs.log` cùng SHA-256 của mô hình.

Thí nghiệm A3 (mục 7.2) minh họa hậu quả: chỉ cần thêm 5 biến sau-đặt, ROC-AUC validation tăng từ 0,783 lên 0,866. Mức tăng này không có thật khi triển khai.

5. Phương pháp

5.1 Hồi quy logistic

Hồi quy logistic mô hình hóa xác suất hủy bằng hàm sigmoid của một tổ hợp tuyến tính các đặc trưng đã biến đổi:

$$p(y = 1 \mid x) = \sigma(z) = 1 / (1 + e^{-z}), \quad z = \beta_0 + \beta_1 x_1 + \ldots + \beta_d x_d$$

Tương đương, **log-odds** là tuyến tính theo đặc trưng: $\log(p / (1 - p)) = z$. Vì vậy $\exp(\beta_j)$ là **odds ratio**: khi x_j tăng một đơn vị và *các biến khác giữ nguyên*, odds hủy nhân với $\exp(\beta_j)$. Tham số được ước lượng bằng cực đại hợp lý, tương đương cực tiểu **log loss** (cross-entropy) có thêm số hạng phạt:

$$L(\beta) = -(1/n) \Sigma_i [\, y_i \log p_i + (1 - y_i) \log(1 - p_i) \,] + (1/C) \cdot R(\beta)$$

trong đó $R(\beta)$ là phạt L2 (tổng bình phương các hệ số, chia 2) hoặc L1 (tổng giá trị tuyệt đối các hệ số), C là nghịch đảo độ mạnh phạt. L1 có thể đưa hệ số về 0 (chọn biến), L2 làm hệ số co đều và ổn định khi các biến tương quan. `class_weight = "balanced"` nhân trọng số lớp hiếm để tăng recall nhưng làm lệch xác suất so với tần suất thật.

Giả định: quan hệ giữa đặc trưng và log-odds là tuyến tính (sau biến đổi), các quan sát độc lập, không có đa cộng tuyến hoàn hảo. Dữ liệu này vi phạm một phần giả định độc lập (booking theo đoàn), nên nhóm dùng bootstrap theo cụm. **Ưu điểm:** cho xác suất có ý nghĩa, dễ hiệu chỉnh, hệ số diễn giải được, huấn luyện nhanh, dễ triển khai. **Nhược**

điểm: không tự học tương tác và quan hệ phi tuyến (cần biến đổi đặc trưng), nhạy với thang đo (cần chuẩn hóa khi có phạt), và hệ số chỉ là liên hệ có điều kiện, không phải nhân quả.

5.2 Pipeline tiền xử lý

Toàn bộ tiền xử lý nằm trong một `sklearn.pipeline.Pipeline` với `ColumnTransformer` gồm ba nhánh, được lưu nguyên khối vào `models/model.joblib`:

- **Phân loại:** `OneHotEncoder(drop="first", min_frequency=50, handle_unknown="infrequent_if_exist")`. Mỗi hệ số được so với một hạng mục tham chiếu; hạng mục hiếm hoặc chưa gặp đi vào nhóm "hiếm" thay vì gây lỗi.
- **Biến đếm:** `log1p` rồi `StandardScaler` (theo kết quả EDA mục 3.4).
- **Nhị phân:** `is_repeated_guest` giữ nguyên 0/1 để odds ratio đọc trực tiếp là khách lặp lại so với khách mới.

Sau biến đổi có 46 cột đầu vào cho mô hình. Thời gian huấn luyện một mô hình L2 (lbfgs) dưới 1 giây; L1 dùng solver liblinear (lbfgs không hỗ trợ L1). Vì scikit-learn 1.9 đã thay tham số `penalty` bằng `l1_ratio`, nhóm dùng `l1_ratio = 0` cho L2 và `1` cho L1.

6. Thiết kế thí nghiệm

6.1 Chia dữ liệu theo thời gian

Bảng 4. *Ba tập chia theo ngày đến* Nguồn: reports/data_quality.md.

Tập	Ngày đến	Số booking	Tỷ lệ hủy	Vai trò
Train	01/07/2015 – 31/12/2016	78.586	36,2%	Học tham số, EDA
Validation	01/01/2017 – 30/04/2017	18.443	36,5%	Chọn tham số, class_weight, ngưỡng
Test	01/05/2017 – 31/08/2017	22.177	40,6%	Đánh giá một lần để kết luận

Chia theo thời gian mô phỏng đúng triển khai: học từ quá khứ, dự đoán booking đến trong tương lai. Đánh đổi là validation (mùa đông–xuân) và test (mùa hè) khác mùa, và tỷ lệ hủy test cao hơn. Mô hình cuối **chỉ fit trên train** để các ngưỡng chọn trên validation khớp đúng với mô hình được lưu (D-11).

6.2 Baseline

- **B0, tỷ lệ hủy chung:** `DummyClassifier(strategy="prior")`, dự đoán 36,2% cho mọi booking. ROC-AUC luôn bằng 0,5; log loss là mốc tối thiểu phải vượt.

- **B1, logistic chỉ với lead time:** biến mạnh nhất trong EDA, chuẩn hóa, không tuning.

6.3 Chọn tham số trên validation

Lưới gồm C ∈ {0,001; 0,01; 0,1; 1; 10; 100} × penalty {L1, L2} × `class_weight` ∈ {None, balanced}, tổng 24 cấu hình. Mỗi cấu hình fit trên train và chấm trên validation. **Tiêu chí chọn là log loss** vì sản phẩm trả về xác suất; ROC-AUC chỉ đo thứ hạng và không phát hiện xác suất bị lệch.

6.4 Ablation

- **A1:** bỏ log1p, chỉ chuẩn hóa biến đếm (kiểm chứng quyết định từ EDA).
- **A2:** thêm 3 biến nghi vấn `deposit_type`, `adr`, `total_of_special_requests` (đo cái giá của việc loại chúng).
- **A3:** thêm 5 biến sau-đặt (minh họa rò rỉ, không bao giờ dùng để triển khai).

Ablation chỉ chạy trên validation và không được dùng để chọn mô hình cuối.

6.5 Khoảng biến thiên và giao thức test

Hồi quy logistic với lbfgs là tất định, nên biến thiên theo seed bằng 0. Biến thiên do mẫu được ước lượng bằng **bootstrap theo cụm ngày đến** (200 lần, seed 42): mỗi lần lấy mẫu lại các ngày đến, giữ nguyên mọi booking của ngày được chọn, vì booking cùng ngày, cùng đoàn tương quan với nhau. Tập test được đánh giá **một lần** sau khi mô hình, đặc trưng và ngưỡng đã commit (commit `b19833a`, 14:34 UTC ngày 11/10/2026). Một lần chạy lại sau đó chỉ để sửa lỗi hiển thị hình; mọi metric giống hệt (D-16).

6.6 Chọn ba ngưỡng hành động

Ba ngưỡng được chọn trên dự đoán validation của mô hình chính: (1) **ưu tiên recall**, ngưỡng cao nhất vẫn bắt được ít nhất 90% booking hủy; (2) **cân bằng**, ngưỡng có F1 lớn nhất; (3) **theo năng lực gọi**, ngưỡng gắn cờ 15% booking có xác suất cao nhất. Ban đầu mục tiêu recall là 80%, nhưng cho ngưỡng gần trùng ngưỡng F1 (0,328 so với 0,335) nên được nâng lên 90%; quyết định này dựa trên validation (D-14).

7. Kết quả

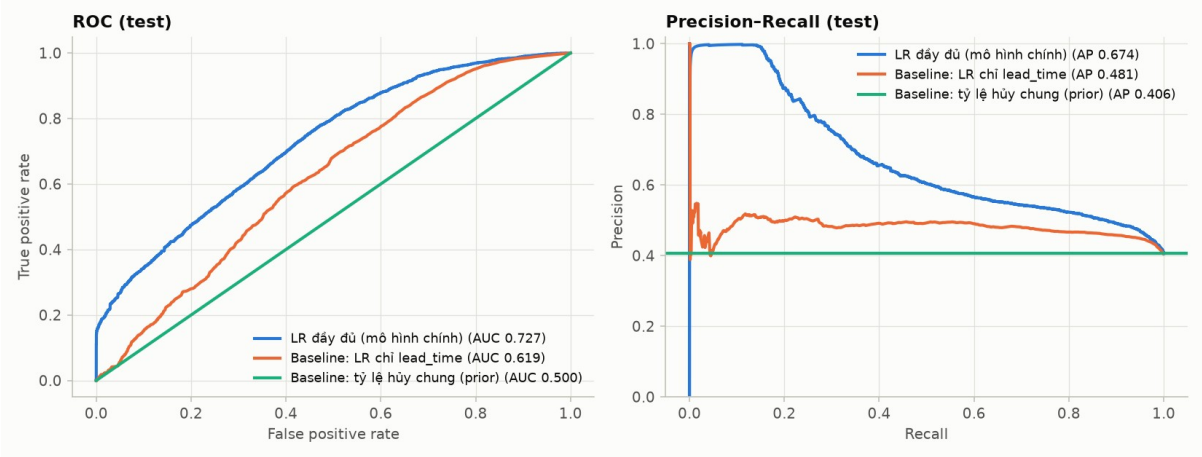
7.1 Thí nghiệm 1: baseline và mô hình đầy đủ

Bảng 5. *So sánh baseline và mô hình đầy đủ trên cùng split (validation để chọn, test để kết luận)*
Nguồn: models/metrics.json, reports/results_comparison.csv.

Mô hình	Tập	ROC-AUC	CI 95%	PR-AUC	Log loss	Brier
Baseline: tỷ lệ hủy chung (prior)	validation	0,500	0,500–0,500	0,365	0,656	0,232
Baseline: tỷ lệ hủy chung (prior)	test	0,500	0,500–0,500	0,406	0,679	0,243

Mô hình	Tập	ROC-AUC	CI 95%	PR-AUC	Log loss	Brier
Baseline: LR chỉ lead_time	validation	0,695	0,676–0,715	0,550	0,617	0,215
Baseline: LR chỉ lead_time	test	0,619	0,603–0,634	0,481	0,679	0,242
LR đầy đủ	validation	0,783	0,757–0,812	0,711	0,527	0,176
LR đầy đủ	test	0,727	0,709–0,748	0,674	0,583	0,200

Mô hình đầy đủ vượt cả hai baseline trên mọi metric, ở cả validation và test; khoảng tin cậy ROC-AUC của mô hình chính (0,709–0,748) không giao với baseline lead time (0,603–0,634). Đáng chú ý, trên test baseline lead time có log loss (0,679) không tốt hơn baseline tỷ lệ chung (0,679): xếp hạng tốt hơn ngẫu nhiên không đảm bảo xác suất tốt hơn khi phân bố thay đổi theo thời gian.



Hình 4. *Đường ROC và Precision–Recall trên test của mô hình chính và hai baseline.* Nguồn: reports/figures/fig_roc_pr.png, src/evaluate.py.

ROC-AUC giảm từ 0,783 (validation) xuống 0,727 (test). Nguyên nhân được phân tích ở mục 8: hiệu năng giảm dần theo tháng, rõ nhất ở tháng 7–8/2017, và tỷ lệ hủy mùa hè cao hơn giai đoạn huấn luyện.

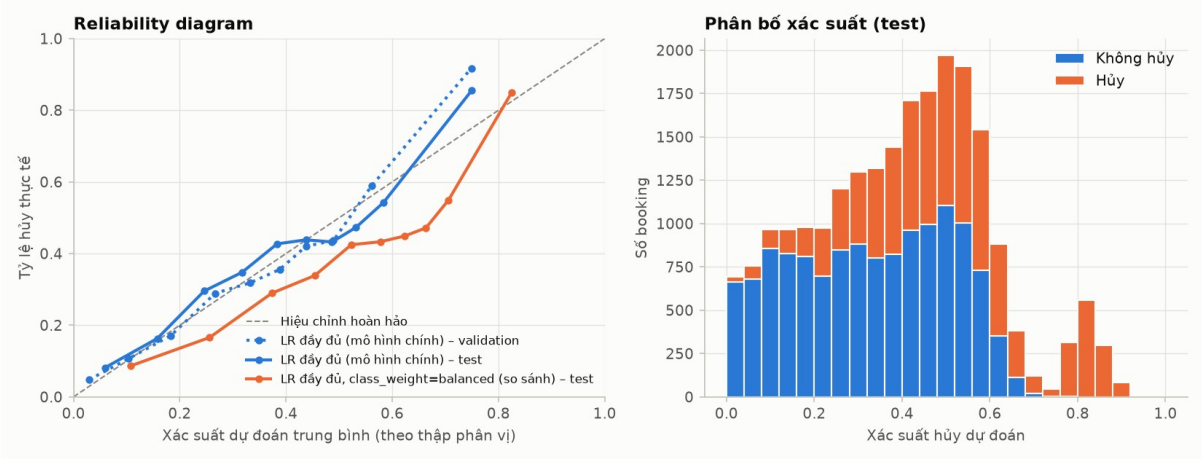
7.2 Chọn tham số và ablation

Bảng 6. *Kết quả chọn tham số và ablation trên validation (rút gọn; đầy đủ ở Phụ lục B)* Nguồn: reports/experiments.csv.

Mã	Cấu hình	ROC-AUC	PR-AUC	Log loss
G00	L2, C = 0,001, class_weight = None	0,7823	0,7045	0,5394
G03	**L2, C = 1, class_weight = None (được chọn)**	**0,7834**	**0,7112**	**0,5269**
G09	L1, C = 1, class_weight = None	0,7833	0,7111	0,5269
G15	L2, C = 1, class_weight = balanced	0,7835	0,7088	0,5520
A1	G03 nhưng bỏ log1p	0,7655	0,6938	0,5436
A2	G03 + deposit_type, adr, special requests	0,8411	0,8044	0,4469
A3	G03 + 5 biến sau-đặt (minh họa rò rỉ)	0,8658	0,7955	0,4367

Với C ≥ 0,01, kết quả gần như không đổi (log loss chênh dưới 0,001): dữ liệu đủ lớn so với 46 cột nên phạt nhẹ là đủ; L1 và L2 tương đương. `class_weight = "balanced"` cho ROC-AUC gần bằng nhưng log loss xấu hơn rõ (0,552 so với 0,527) vì xác suất bị đẩy lên. Ablation A1 xác nhận log1p giúp ích (ROC-AUC 0,766 so với 0,783). A2 cho thấy cái giá của nguyên tắc bảo thủ: thêm ba biến nghi vấn tăng ROC-AUC lên 0,841, nhưng nhóm không dùng vì không chắc chúng có ngay lúc đặt.

7.3 Thí nghiệm 2: calibration



Hình 5. *Reliability diagram (validation, test và mô hình class_weight = balanced) và phân bố xác suất dự đoán trên test.* Nguồn: reports/figures/fig_calibration.png.

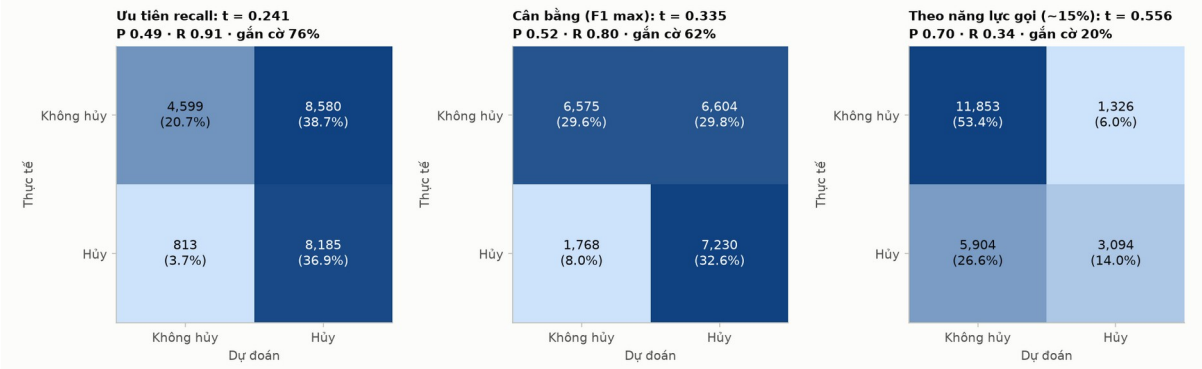
Trên test, log loss 0,583 và Brier 0,200. Xác suất dự đoán trung bình 39,5% so với tỷ lệ hủy thực tế 40,6%: lệch −1,1 điểm phần trăm, tức hiệu chỉnh tổng thể tốt dù tỷ lệ hủy test cao hơn train. Đường reliability của mô hình chính bám đường chéo ở vùng 0–0,6; ở thập phân vị cao nhất, tỷ lệ hủy thực tế (khoảng 0,85) cao hơn dự đoán (0,75), tức mô hình hơi thận trọng với các booking rủi ro rất cao. Mô hình `class_weight = "balanced"` nằm dưới đường chéo ở mọi vùng (dự đoán quá cao), log loss 0,611 và Brier 0,214. **Kết luận:** mô hình chính được hiệu chỉnh tốt để dùng xác suất trực tiếp; không cần hiệu chỉnh thêm (Platt/isotonic), nhưng cần theo dõi khi tỷ lệ hủy thay đổi.

Log loss đo gì khác accuracy? Accuracy chỉ đếm số dự đoán đúng sau khi cắt ở một ngưỡng, nên hai mô hình cho xác suất 0,51 và 0,99 với cùng một booking hủy là như nhau. Log loss phạt theo −log của xác suất gán cho kết quả thật: tự tin mà sai bị phạt rất nặng, đúng mà thiếu tự tin cũng bị phạt. Vì vậy log loss đánh giá chất lượng của chính con số xác suất, đúng với đầu ra của sản phẩm.

7.4 Thí nghiệm 3: ba ngưỡng hành động

Bảng 7. *Ba ngưỡng chọn trên validation và kết quả áp dụng nguyên trạng trên test* Nguồn: models/metrics.json.

Ngưỡng	t	Precision val → test	Recall val → test	Gắn cờ val → test	TP / FP / FN / TN (test)
Ưu tiên recall (recall ≥ 90% trên val)	0,241	0,480 → 0,488	0,900 → 0,910	68,5% → 75,6%	8.185 / 8.580 / 813 / 4.599
Cân bằng (F1 lớn nhất trên val)	0,335	0,529 → 0,523	0,793 → 0,804	54,8% → 62,4%	7.230 / 6.604 / 1.768 / 6.575
Theo năng lực gọi (~15%)	0,556	0,808 → 0,700	0,332 → 0,344	15,0% → 19,9%	3.094 / 1.326 / 5.904 / 11.853



Hình 6. *Ma trận nhầm lẫn trên test tại ba ngưỡng hành động.* Nguồn: reports/figures/fig_confusion_thresholds.png.

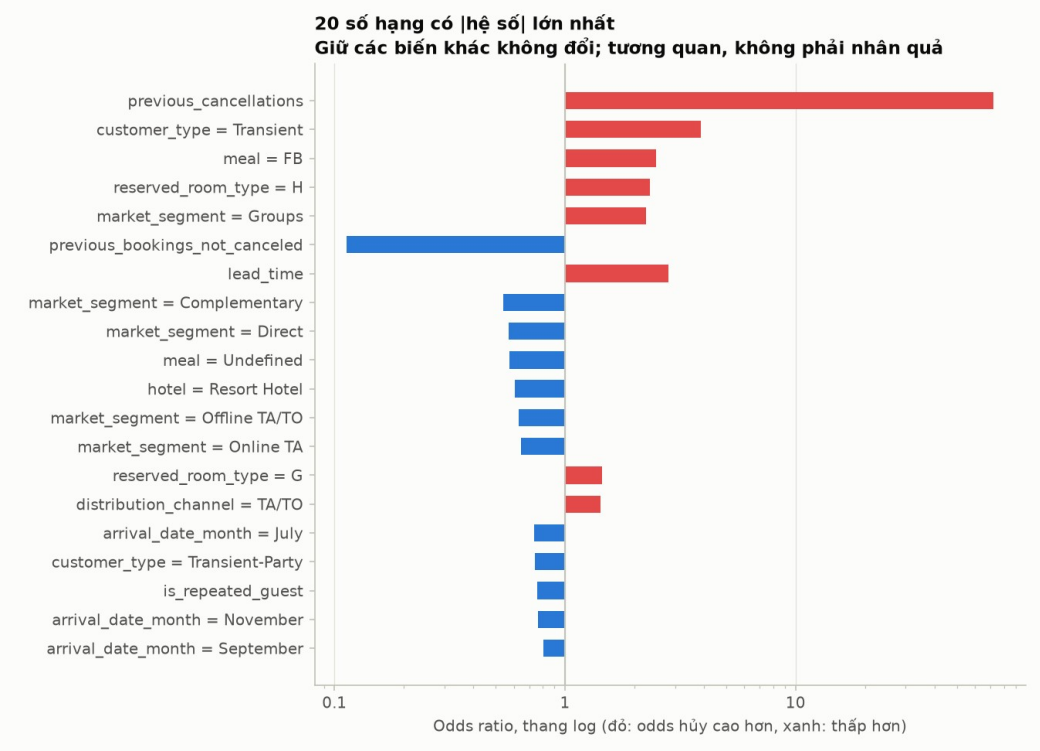
Precision và recall trên test rất gần validation ở hai ngưỡng đầu, cho thấy ngưỡng chuyển giao tốt. Ở ngưỡng năng lực gọi, tỷ lệ gắn cờ tăng từ 15% lên 19,9% và precision giảm từ 0,81 xuống 0,70: mùa hè có nhiều booking xác suất cao hơn. Nếu năng lực gọi cố định ở 15%, khách sạn nên xếp hạng và gọi đúng 15% booking đầu danh sách mỗi ngày thay vì dùng ngưỡng xác suất cố định. Trong ứng dụng, booking có p ≥ 0,556 là **Rất cao**, ≥ 0,335 là **Cao**, ≥ 0,241 là **Trung bình**, còn lại là **Thấp**.

7.5 Thí nghiệm 4: hệ số và odds ratio

Bảng 8. *Mười hai số hạng có |hệ số| lớn nhất (mô hình fit trên train)* Nguồn: reports/odds_ratios.csv.

Số hạng	So sánh	Hệ số	Odds ratio
previous_cancellations	0 → 1 (log1p, chuẩn hóa)	1,468	71,943
customer_type = Transient	so với customer_type = Contract	1,359	3,892
meal = FB	so với meal = BB	0,907	2,477
reserved_room_type = H	so với reserved_room_type = A	0,848	2,336
market_segment = Groups	so với market_segment = Aviation	0,812	2,252
previous_bookings_not_canceled	0 → 1 (log1p, chuẩn hóa)	-0,774	0,113
lead_time	16 → 158 (log1p, chuẩn hóa)	0,753	2,800
market_segment = Complementary	so với market_segment = Aviation	-0,617	0,539
market_segment = Direct	so với market_segment = Aviation	-0,560	0,571
meal = Undefined	so với meal = BB	-0,554	0,574
hotel = Resort Hotel	so với hotel = City Hotel	-0,503	0,605

Số hạng	So sánh	Hệ số	Odds ratio
market_segment = Offline TA/TO	so với market_segment = Aviation	-0,459	0,632



Hình 7. *Odds ratio của 20 số hạng có |hệ số| lớn nhất (thang log).* Nguồn: reports/figures/fig_odds_ratios.png.

Odds ratio của biến số được tính cho thay đổi từ phân vị 25 đến 75 của train (hoặc từ trung vị đến trung vị + 1), qua biến đổi log1p và chuẩn hóa; biến phân loại so với hạng mục tham chiếu. Diễn giải **có điều kiện**, ví dụ:

- *Giữ các biến khác không đổi*, booking có lead time 158 ngày có odds hủy gấp khoảng **2,8 lần** booking có lead time 16 ngày.
- *Giữ các biến khác không đổi*, khách đã từng có 1 booking hủy trước đó có odds hủy gấp khoảng **72 lần** khách chưa từng hủy. Con số rất lớn này phản ánh mẫu bất thường của booking đoàn (mục 3.4), không phải quy luật hành vi chung.
- *Giữ các biến khác không đổi*, mỗi booking trước không hủy (0 → 1) đi kèm odds hủy chỉ bằng khoảng **0,11 lần**; booking ở Resort Hotel có odds bằng 0,61 lần City Hotel.

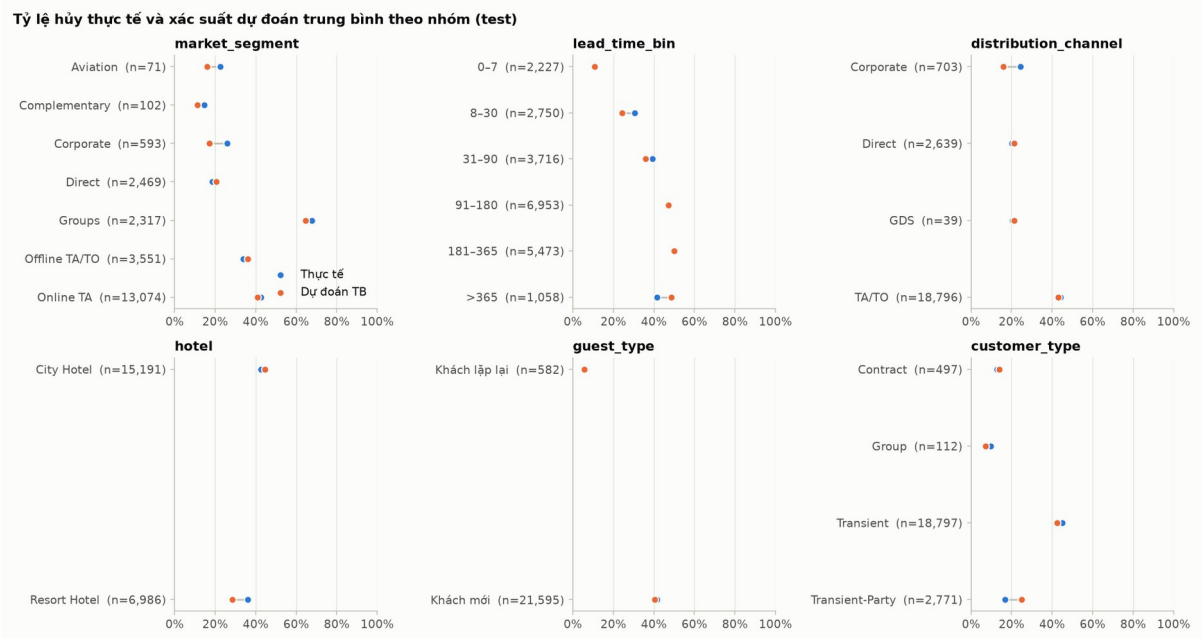
Odds ratio có phải tác động nhân quả không? Không. Đây là liên hệ thống kê có điều kiện trong dữ liệu quan sát, chịu ảnh hưởng của biến gây nhiễu không đo được (mục đích chuyến đi, chính sách giá), tương quan giữa các đặc trưng (market_segment và distribution_channel) và cách chọn hạng mục tham chiếu. Ví dụ: hệ số dương của `customer_type = Transient` không có nghĩa là đổi một khách sang hợp đồng sẽ làm khách ít hủy hơn. Muốn biết tác động nhân quả cần thí nghiệm có kiểm soát (ví dụ A/B test một chính sách nhắc xác nhận).

8. Phân tích lỗi

8.1 Theo nhóm

Bảng 9. *Hiệu năng trên test theo nhóm (ngưỡng cân bằng t = 0,335)* Nguồn: reports/error_by_group.csv.

Nhóm	n	Hủy thực tế	Dự đoán TB	ROC-AUC	Precision	Recall	Gắn cờ
City Hotel	15.191	42,6%	44,6%	0,739	0,518	0,873	71,8%
Resort Hotel	6.986	36,2%	28,5%	0,733	0,540	0,626	42,0%
Online TA	13.074	42,7%	41,0%	0,632	0,499	0,839	71,8%
Offline TA/TO	3.551	34,0%	36,2%	0,766	0,487	0,767	53,5%
Groups	2.317	67,7%	64,6%	0,901	0,742	0,928	84,6%
Direct	2.469	18,7%	20,5%	0,623	0,242	0,258	19,9%
Corporate	593	25,8%	17,3%	0,799	0,505	0,301	15,4%
Lead time 0–7	2.227	11,1%	10,8%	0,640	0,886	0,126	1,6%
Lead time 8–30	2.750	30,6%	24,4%	0,598	0,353	0,208	18,0%
Lead time 31–90	3.716	39,4%	35,9%	0,574	0,438	0,709	63,8%
Lead time 91–180	6.953	47,0%	47,1%	0,717	0,534	0,942	83,0%
Lead time 181–365	5.473	50,0%	50,0%	0,705	0,561	0,939	83,7%
Lead time > 365	1.058	41,5%	48,6%	0,792	0,580	0,768	54,9%
Khách mới	21.595	41,5%	40,4%	0,718	0,523	0,805	63,9%
Khách lặp lại	582	6,4%	5,8%	0,751	0,517	0,405	5,0%

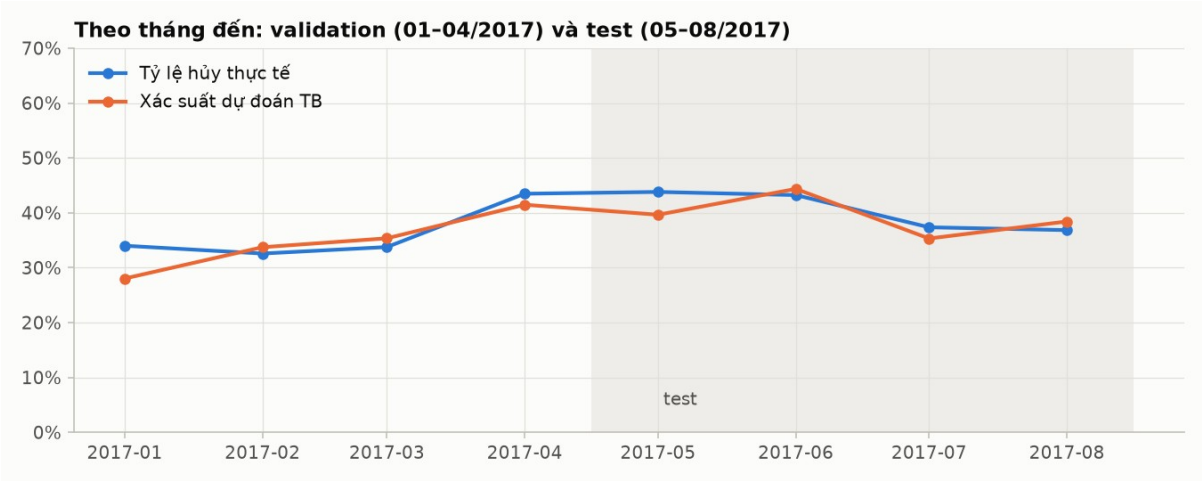


Hình 8. *Tỷ lệ hủy thực tế (xanh) và xác suất dự đoán trung bình (cam) theo nhóm trên test.* Nguồn: reports/figures/fig_error_groups.png.

- **Phần mở rộng, City và Resort:** khả năng xếp hạng tương đương (ROC-AUC 0,739 và 0,733), nhưng mô hình **dự đoán thấp** cho Resort (28,5% so với 36,2% thực tế) nên recall ở Resort chỉ 0,63 so với 0,87 ở City. Một mô hình gộp với hiệu ứng `hotel` cố định không bắt được việc tỷ lệ hủy Resort mùa hè 2017 tăng.

- **Online TA**, nhóm lớn nhất (59% test), có ROC-AUC thấp nhất (0,632) trong các nhóm lớn; ngược lại *Groups* đạt 0,901. Phần lớn khả năng phân biệt của mô hình đến từ lead time dài, phân khúc đoàn và lịch sử hủy, còn booking cá nhân qua OTA khó phân biệt bằng thông tin lúc đặt.
- **Lead time 8–90 ngày** có ROC-AUC chỉ 0,57–0,60: trong khoảng này lead time không còn tách lớp tốt.
- **Khách lặp lại** ít hủy (6,4%) và được dự đoán đúng mức (5,8%); chỉ 5% bị gắn cờ so với 64% ở khách mới.

8.2 Theo thời gian đến



Hình 9. *Tỷ lệ hủy thực tế và xác suất dự đoán trung bình theo tháng đến, validation và test.* Nguồn: reports/figures/fig_monthly.png.

Bảng 10. *Hiệu năng theo tháng đến trên test* Nguồn: reports/error_by_group.csv.

Tháng	n	Hủy thực tế	Dự đoán TB	ROC-AUC	Recall
2017-05	6.305	43,8%	39,6%	0,782	0,800
2017-06	5.639	43,2%	44,4%	0,756	0,879
2017-07	5.310	37,4%	35,3%	0,675	0,719
2017-08	4.923	36,8%	38,4%	0,666	0,799

Mức xác suất trung bình theo tháng bám tỷ lệ thực tế khá sát (sai lệch dưới 4 điểm phần trăm), nhưng **khả năng xếp hạng giảm dần** từ 0,782 (tháng 5) xuống 0,666 (tháng 8), tức càng xa giai đoạn huấn luyện mô hình càng kém. Đây là bằng chứng trôi theo thời gian; khi triển khai cần đánh giá hằng tháng và huấn luyện lại định kỳ.

8.3 Các trường hợp FP/FN điển hình

Bảng 11. *Năm FP và năm FN mà mô hình tin chắc nhất (test, ngưỡng cân bằng)* Nguồn: reports/fp_fn_examples.csv.

Loại	p	Khách sạn	Lead time	Phân khúc	Hủy trước	Khách lặp lại
FP	0,983	Resort Hotel	80	Direct	1	không
FP	0,845	City Hotel	208	Groups	0	không

Loại	p	Khách sạn	Lead time	Phân khúc	Hủy trước	Khách lặp lại
FP	0,797	City Hotel	174	Groups	0	không
FP	0,796	Resort Hotel	178	Online TA	0	không
FP	0,773	City Hotel	12	Corporate	1	có
FN	0,000	City Hotel	1	Corporate	0	có
FN	0,001	Resort Hotel	1	Direct	0	có
FN	0,002	Resort Hotel	0	Direct	0	có
FN	0,002	City Hotel	4	Corporate	0	có
FN	0,002	City Hotel	4	Complementary	0	có

- **FP mạnh nhất** (p = 0,983) là booking Direct có `previous_cancellations = 1`: mô hình áp quy luật "đã từng hủy một lần thì gần như chắc chắn hủy" (học từ booking đoàn) cho một khách cá nhân. Các FP còn lại là booking *Groups* đặt sớm 170–210 ngày nhưng vẫn đến.
- **FN tin chắc nhất** (p ≈ 0) đều là khách lặp lại, lead time 0–4 ngày, kênh Corporate/Direct: nhóm hiếm khi hủy nên mô hình gán xác suất rất thấp, nhưng một số vẫn hủy (có thể vì thay đổi lịch công tác).
- Cơ cấu lỗi: 71% FP và 51% FN thuộc Online TA; FN có tỷ trọng Direct cao hơn hẳn nhóm dự đoán đúng (19,4% so với 12,7%). Booking `Non Refund` (biến không được dùng) chiếm 14,6% nhóm dự đoán đúng nhưng chỉ 0,8% FN: mô hình vẫn nhận ra phần lớn chúng qua lead time, phân khúc và lịch sử hủy.

8.4 Trường hợp ngoài miền

Chỉ 11 booking test có giá trị số nằm ngoài khoảng [min, max] của train (đều không hủy, xác suất dự đoán trung bình 0,066), và không có hạng mục mới. Mô hình tuyến tính trên log-odds vẫn ngoại suy khi đầu vào vượt miền: với một booking điển hình, lead time 629 ngày cho p = 0,702, còn 1.000 ngày và 2.000 ngày cho 0,745 và 0,801, dù train không có booking nào trên 737 ngày. Vì vậy API trả **cảnh báo** khi đầu vào nằm ngoài miền train hoặc ngoài khoảng 1–99% (mục 9).

9. Ứng dụng web và API

9.1 Kiến trúc

Backend dùng **FastAPI** (Python), không dùng cơ sở dữ liệu. Huấn luyện chạy offline (`src/train.py`); khi khởi động, app chỉ nạp `models/model.joblib` và `config.json`, không huấn luyện lại ở mỗi request. Các endpoint GET **chỉ đọc file đã sinh** (`models/metrics.json`, `reports/curves.json`, `reports/*.csv`, model card), không tính lại. App không lưu và không ghi log nội dung booking; handler lỗi 500 chỉ ghi đường dẫn và trả thông điệp an toàn.

Giao diện viết bằng **React + TypeScript** (Vite, Tailwind CSS v4, shadcn/ui trên Base UI, TanStack Query/Table, React Hook Form + Zod, Recharts, motion), build sẵn vào `app/web` và được FastAPI phục vụ cùng cổng, nên `make app` chạy được ngay mà không cần Node.js. UI chỉ gọi API, không chứa logic mô hình; mọi số liệu và biểu đồ lấy từ API, thiếu thì hiển thị trạng thái rỗng thay vì dữ liệu giả.

Bảng 12. *Các endpoint* Nguồn: app/main.py.

Endpoint	Mô tả
`POST /api/cancellation-risk`	Nhận booking (JSON), trả `probability`, `risk_group`, `action`, `threshold_used`, `thresholds`, `model_version`, `warnings`
`POST /api/explain`	Cùng đầu vào; trả hệ số chặn, đóng góp log-odds của từng biến và đường xác suất theo lead time
`GET /api/model-info`	Phiên bản, ngưỡng, danh sách trường đầu vào (kiểu, đơn vị, miền cho phép, miền train, hạng mục hợp lệ), booking mẫu, biến đã loại
`GET /api/metrics`	Metric validation/test, đường ROC/PR, reliability theo bin, đường precision/recall theo ngưỡng, odds ratio, sai số theo nhóm
`GET /api/model-card`, `/api/data-card`	Model card (markdown) và data card (nguồn, checksum, chất lượng, data dictionary)
`GET /api/health`, `/api/files/…`, `/docs`	Kiểm tra kết nối; tải metrics.json, data_dictionary.csv, model_card.md; tài liệu OpenAPI kèm ví dụ

9.2 Kiểm tra đầu vào

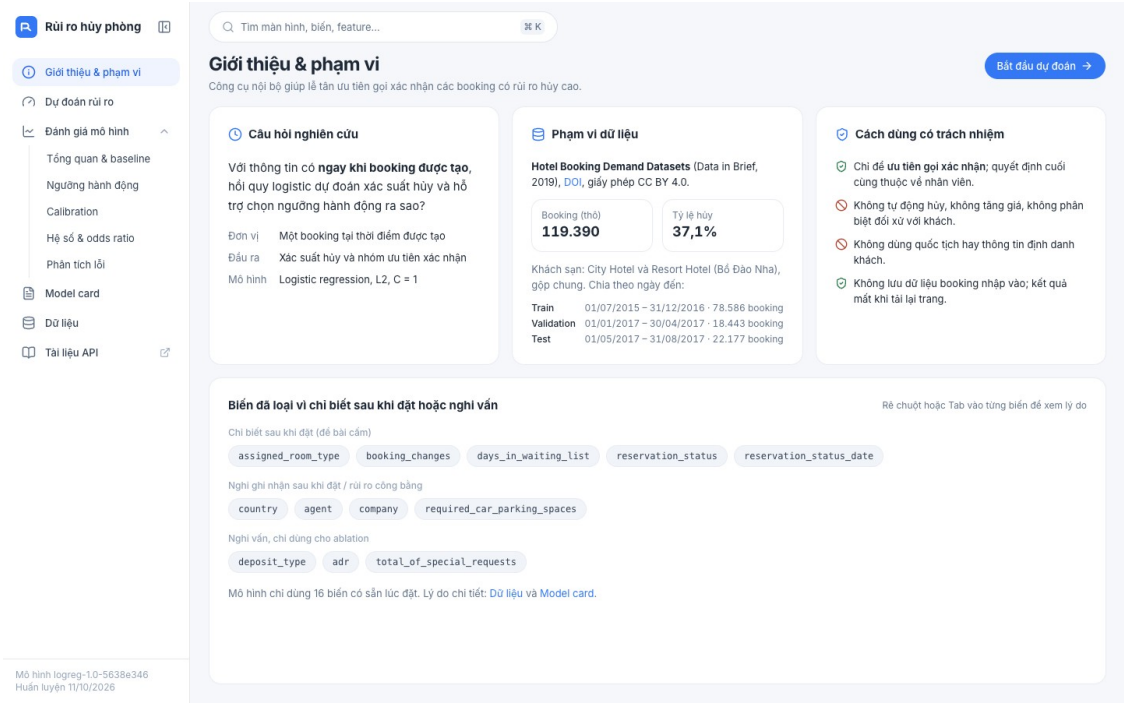
Schema Pydantic (`app/schemas.py`) dùng **chế độ strict**: không tự ép kiểu (chuỗi "12" không được hiểu là 12), không cho trường lạ (gửi `country` bị từ chối), có miền giá trị cho từng trường số và danh sách hạng mục hợp lệ khớp với dữ liệu train (có test kiểm tra). Form trên giao diện dựng schema Zod từ `/api/model-info` nên dùng cùng miền giá trị, báo lỗi ngay dưới ô và không gửi khi thiếu trường hoặc sai kiểu. Giá trị hợp lệ nhưng ngoài miền train vẫn được gửi, kèm nhãn "ngoài miền" và cảnh báo từ API. Không giá trị nào bị âm thầm thay thế.

Bảng 13. *Mã lỗi của API* Nguồn: app/main.py, tests/test_api.py.

Tình huống	Mã	Ví dụ thông điệp
Thiếu trường hoặc body không phải JSON	400	hotel: Thiếu trường bắt buộc
Sai kiểu	422	lead_time: Phải là số nguyên
Hạng mục không hợp lệ / ngoài miền	422	market_segment: Giá trị không hợp lệ; lead_time: Phải ≤ 1000
Trường lạ; tổng số khách = 0	422	Trường không được hỗ trợ; Tổng số khách phải ≥ 1
Hợp lệ nhưng hiếm / ngoài miền train	200 + warnings	lead_time = 900 nằm ngoài miền dữ liệu train [0, 737]…
Lỗi nội bộ	500	Lỗi nội bộ; không có dữ liệu nào được lưu

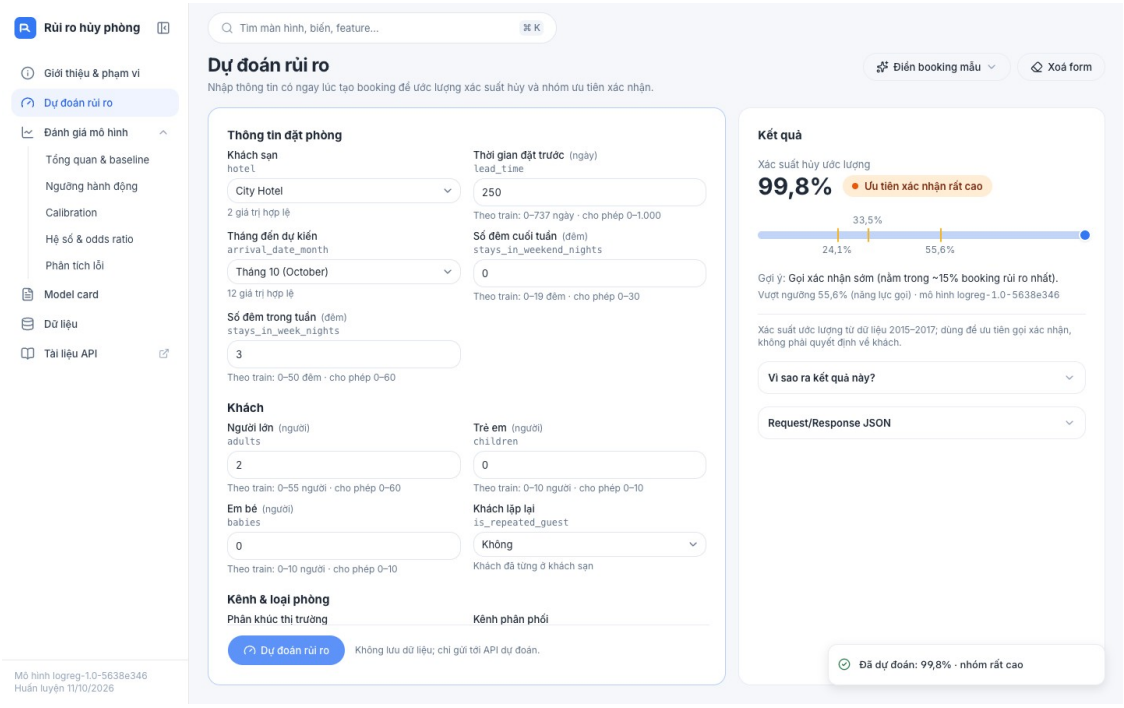
9.3 Giao diện

Bố cục kiểu bento: sidebar dạng cây (thu thành cột icon ở màn hình vừa, thành ngăn kéo trên điện thoại), ô tìm kiếm toàn cục `Ctrl/⌘ K` tìm không dấu theo tên màn hình và tên biến (nhảy tới đúng dòng trong bảng odds ratio hoặc data dictionary). Từ 900 × 860 px trở lên mỗi trang vừa một màn hình, bảng cuộn bên trong khung. Có năm nhóm màn hình: Giới thiệu & phạm vi, Dự đoán rủi ro, Đánh giá mô hình (tổng quan & baseline, ngưỡng, calibration, odds ratio, phân tích lỗi), Model card (render nguyên văn `reports/model_card.md`) và Dữ liệu.

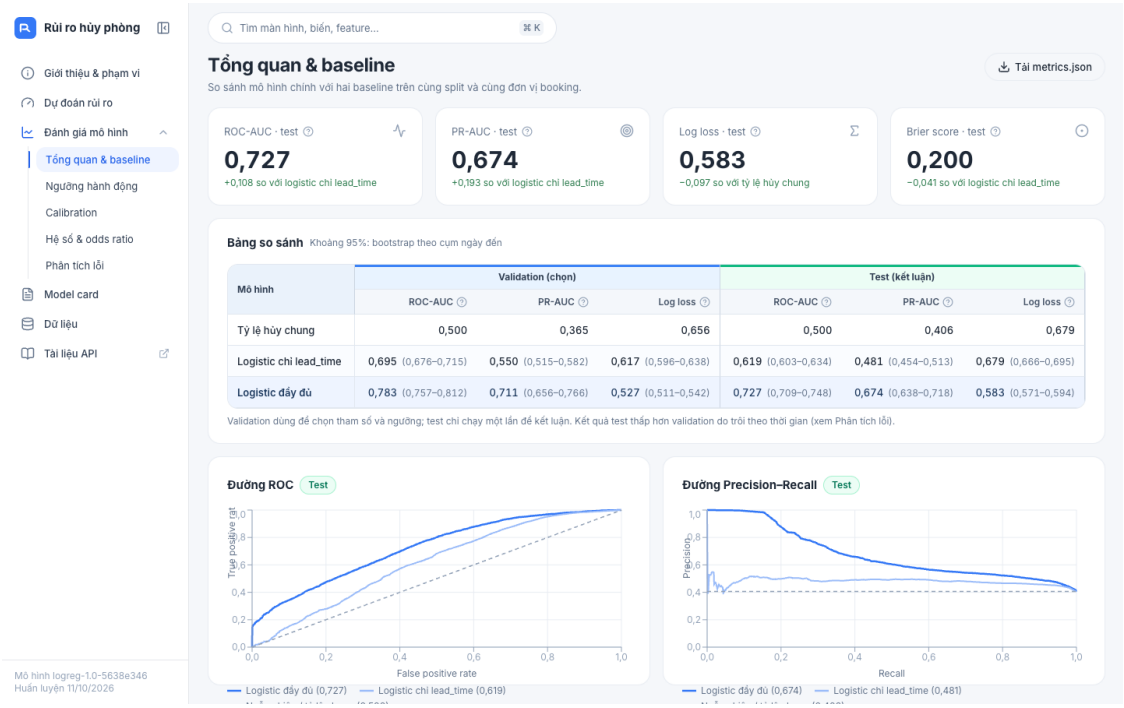


Hình 10. *Màn hình giới thiệu: câu hỏi nghiên cứu, phạm vi dữ liệu, cách dùng có trách nhiệm và các biến đã loại (kèm lý do khi rê chuột).* Nguồn: ảnh chụp ứng dụng (frontend/src/pages/intro.tsx).

Màn hình **dự đoán** chia form thành bốn nhóm (thông tin đặt phòng, khách, kênh & loại phòng, lịch sử); mỗi trường có nhãn tiếng Việt kèm tên biến, đơn vị và miền giá trị theo train. Khung kết quả gồm xác suất (đếm lên), nhóm ưu tiên xác nhận, thanh xác suất với ba vạch ngưỡng, cảnh báo từ API, dòng chú thích cố định, phần JSON request/response có nút sao chép và phần giải thích đóng góp log-odds. Ba booking mẫu giả lập (rủi ro thấp/trung bình/cao) và một mẫu ngoài miền lấy từ backend.



Hình 11. *Màn hình dự đoán với booking mẫu rủi ro cao: xác suất 99,8%, nhóm ưu tiên xác nhận rất cao, thanh ba ngưỡng.* Nguồn: ảnh chụp ứng dụng (frontend/src/pages/predict.tsx).



Hình 12. *Màn hình tổng quan & baseline: bốn KPI trên test so với baseline tốt nhất, bảng so sánh hai cấp Validation/Test, đường ROC và PR.* Nguồn: ảnh chụp ứng dụng (frontend/src/pages/evaluation.tsx).

9.4 Kiểm thử

Bộ test gồm 51 test chạy bằng `make test`: 6 test leakage, 7 test tiền xử lý (chia theo thời gian, làm sạch, pipeline chỉ học từ train, hạng mục chưa gặp, ngưỡng) và 38 test API (request hợp lệ, nhóm rủi ro khớp ngưỡng, thiếu trường → 400, sai kiểu → 422, ngoài miền → 422, trường lạ, JSON hỏng, cảnh báo, lỗi 500 an toàn, schema khớp dữ liệu train, phép tách log-odds chính xác, booking mẫu đúng nhóm, endpoint chỉ đọc, chỉ

tải được tệp trong danh sách cho phép, route giao diện không lộ tệp ngoài thư mục build).

Ngoài ra, `npm run e2e` (thư mục `frontend`) điều khiển Chrome headless qua 17 kịch bản: điền booking mẫu → dự đoán → xem nhóm ưu tiên → sang dashboard → model card; input xấu (để trống, chữ trong ô số, số âm, số thập phân, giá trị ngoài miền, hạng mục không hợp lệ gửi thẳng qua API) không làm sập app và báo lỗi đúng chỗ; tìm kiếm không dấu; ngăn kéo điều hướng ở 390 px. Kịch bản cũng kiểm tra không có lỗi console và trang không cuộn ở 1440 × 900.

10. Đạo đức và giới hạn

10.1 Sử dụng có trách nhiệm

Mô hình **chỉ dùng để ưu tiên gọi xác nhận**. Model card (`reports/model_card.md`) và giao diện (trang giới thiệu, dòng chú thích ở kết quả dự đoán) đều nêu các sử dụng bị cấm: tự động hủy hay từ chối khách, tăng giá hoặc đòi đặt cọc dựa trên điểm rủi ro, phân biệt đối xử theo bất kỳ đặc điểm nào. Kết quả gắn cờ chỉ dẫn tới một cuộc gọi, không dẫn tới bất lợi cho khách; quyết định cuối cùng thuộc về nhân viên.

10.2 Công bằng và quyền riêng tư

- Không dùng quốc tịch: ngoài nghi vấn rò rỉ, biến này có thể dẫn tới đối xử khác nhau theo quốc tịch.
- Tỷ lệ gắn cờ khác nhau giữa các nhóm (khách mới 64%, khách lặp lại 5%; City 72%, Resort 42% ở ngưỡng cân bằng) chủ yếu do tỷ lệ hủy thực tế khác nhau. Vì hậu quả của việc bị gắn cờ chỉ là một cuộc gọi, sự chênh lệch này chấp nhận được; nếu hành động đi kèm có tính bất lợi, cần đánh giá lại công bằng.
- Dữ liệu đã ẩn danh; ứng dụng không lưu dữ liệu nhập; booking mẫu trong app là giả lập.

10.3 Giới hạn

1. Chỉ 2 khách sạn ở Bồ Đào Nha, booking 2015–2017; không đại diện cho khách sạn, thị trường hay giai đoạn khác (ví dụ sau đại dịch).
2. Hiệu năng giảm theo thời gian (ROC-AUC 0,78 → 0,73, tháng 8/2017 chỉ 0,67); cần đánh giá và huấn luyện lại định kỳ.
3. Dự đoán thấp cho Resort Hotel; xếp hạng yếu với Online TA và lead time 8–90 ngày.
4. Phụ thuộc mạnh vào `previous_cancellations`, biến có mẫu bất thường do booking đoàn.

5. Loại các biến nghi rò rỉ làm mất khoảng 0,06 ROC-AUC (ablation A2); nếu khách sạn xác nhận chúng có ngay lúc đặt, có thể huấn luyện lại. Ngược lại, số đêm và số khách vẫn có rủi ro đã bị sửa sau lúc đặt trong dữ liệu lịch sử.
6. Mô hình tuyến tính không học tương tác (ví dụ lead time × mùa); mô hình cây có thể xếp hạng tốt hơn nhưng nằm ngoài chủ đề Bài 5 và khó diễn giải hơn.

11. Kết luận

Trả lời câu hỏi nghiên cứu: chỉ với 16 thông tin có ngay lúc đặt, hồi quy logistic ước lượng xác suất hủy với ROC-AUC 0,727 và log loss 0,583 trên tập test độc lập, vượt rõ baseline lead time (0,619) và baseline tỷ lệ chung. Xác suất được hiệu chỉnh tốt ở mức tổng thể, nên có thể dùng trực tiếp để chọn ngưỡng hành động. Ba ngưỡng chọn trên validation chuyển giao tốt sang test với hai mục tiêu bắt nhiều booking hủy (recall 0,91 hoặc 0,80); ngưỡng theo năng lực gọi cần hiệu chỉnh theo mùa vì tỷ lệ gắn cờ tăng từ 15% lên 20%.

Bài học quan trọng nhất là **thời điểm dự đoán quyết định tập đặc trưng**: các biến như chỗ đậu xe, loại tiền cọc hay trạng thái đặt phòng làm điểm số tăng mạnh (đến ROC-AUC 0,87) nhưng không thể có khi booking vừa được tạo. Nhóm chấp nhận điểm thấp hơn để có mô hình dùng được thật. Hệ số và odds ratio giúp hiểu mô hình (lead time dài, lịch sử hủy, phân khúc đoàn đi kèm rủi ro cao) nhưng chỉ là liên hệ có điều kiện, không phải nhân quả.

Hướng phát triển: đánh giá cuốn chiếu theo tháng và huấn luyện lại định kỳ; xác minh với khách sạn thời điểm ghi nhận của `deposit_type`, `adr`, yêu cầu đặc biệt; thêm tương tác (lead time × tháng, khách sạn × tháng) hoặc mô hình riêng cho từng khách sạn; thử nghiệm A/B tác động của việc gọi xác nhận lên tỷ lệ hủy thật.

12. Tài liệu tham khảo

[1] Nguyễn Văn Hậu. *Slide học phần Học máy cơ bản, Bài 5: Hồi quy logistic, phân loại bằng xác suất*. Lớp 12523W.1.

[2] Antonio, N., de Almeida, A., & Nunes, L. (2019). Hotel booking demand datasets. *Data in Brief*, 22, 41–49. Trang dữ liệu: https://pmc.ncbi.nlm.nih.gov/articles/PMC6297060/

[3] DOI bài mô tả dữ liệu: https://doi.org/10.1016/j.dib.2018.11.126 (giấy phép CC BY 4.0).

[4] scikit-learn developers. *Linear Models: Logistic regression*. https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[6] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 625–632.

[7] Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.

[8] Mitchell, M., et al. (2019). Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 220–229.

[9] FastAPI documentation. https://fastapi.tiangolo.com/

Mọi bảng và hình trong báo cáo được sinh từ mã nguồn của nhóm (`src/`, `notebooks/eda.ipynb`) trên dữ liệu [2]; nguồn cụ thể ghi ở chú thích từng bảng/hình.

Phụ lục A. Hướng dẫn tái lập

Yêu cầu Python 3.14 (đã kiểm thử 3.14.7). Trên máy mới:

```
git clone https://github.com/Nguyenvan01/hotel-booking-cancellation-prediction.git && cd hotel-booking-cancellation-prediction
python3.14 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
make all      # = make data train evaluate test app
```

Bảng 14. *Môi trường và cấu hình* Nguồn: models/config.json, config.yaml.

Mục	Giá trị
Python / numpy / pandas / scikit-learn	3.14.7 / 2.5.3 / 3.0.6 / 1.9.1
Seed	42 (config.yaml)
SHA-256 mmc2.zip	6db57d635b9edec05f1cf9616ba6473e864909e4fc34819fa21cff58d518ee6a
SHA-256 H1.csv	6f38fda13794fd6cf6547d2432ffb79666200000eb45fd09e4b97fe49519a6ab
SHA-256 H2.csv	5107090ffdc5585a9e18299801e9d85c3b459c235542882183a502a73ef63c4d
Phiên bản mô hình	logreg-1.0-5638e346
Lần chạy test chính thức	2026-10-11T14:34:24+00:00, model_sha256 = 5638e346…, commit b19833a

Phụ lục B. Bảng theo dõi thí nghiệm (validation)

Bảng 15. *Toàn bộ 29 thí nghiệm* Nguồn: reports/experiments.csv.

Mã	Giai đoạn	C	Penalty	class_weight	ROC-AUC	PR-AUC	Log loss
B0	baseline	–	–	–	0,5000	0,3652	0,6564
B1	baseline	–	–	–	0,6952	0,5503	0,6175
G00	grid	0,001	L2	None	0,7823	0,7045	0,5394
G01	grid	0,01	L2	None	0,7833	0,7111	0,5281
G02	grid	0,1	L2	None	0,7833	0,7112	0,5270
G03 ✓	grid	1	L2	None	0,7834	0,7112	0,5269
G04	grid	10	L2	None	0,7832	0,7110	0,5270
G05	grid	100	L2	None	0,7832	0,7110	0,5270
G06	grid	0,001	L1	None	0,7684	0,6646	0,5586
G07	grid	0,01	L1	None	0,7840	0,7126	0,5291
G08	grid	0,1	L1	None	0,7834	0,7113	0,5272
G09	grid	1	L1	None	0,7833	0,7111	0,5269
G10	grid	10	L1	None	0,7833	0,7111	0,5269
G11	grid	100	L1	None	0,7833	0,7111	0,5269
G12	grid	0,001	L2	balanced	0,7827	0,7029	0,5555
G13	grid	0,01	L2	balanced	0,7836	0,7090	0,5515
G14	grid	0,1	L2	balanced	0,7836	0,7087	0,5521
G15	grid	1	L2	balanced	0,7835	0,7088	0,5520
G16	grid	10	L2	balanced	0,7836	0,7089	0,5519
G17	grid	100	L2	balanced	0,7836	0,7089	0,5519
G18	grid	0,001	L1	balanced	0,7683	0,6666	0,5705
G19	grid	0,01	L1	balanced	0,7844	0,7109	0,5496
G20	grid	0,1	L1	balanced	0,7837	0,7088	0,5514
G21	grid	1	L1	balanced	0,7838	0,7090	0,5518
G22	grid	10	L1	balanced	0,7838	0,7089	0,5520
G23	grid	100	L1	balanced	0,7838	0,7089	0,5520
A1	ablation	1	L2	None	0,7655	0,6938	0,5436
A2	ablation	1	L2	None	0,8411	0,8044	0,4469
A3	ablation	1	L2	None	0,8658	0,7955	0,4367

Mô tả A1–A3 ở mục 6.4. Dấu ✓ là cấu hình được chọn (log loss validation nhỏ nhất).

Phụ lục C. Từ điển dữ liệu

Bảng 16. *Vai trò và thời điểm có sẵn của các biến* Nguồn: data/data_dictionary.csv.

Biến	Kiểu	Vai trò	Thời điểm có sẵn
is_canceled	int (0/1)	target	sau đặt (nhãn)
hotel	category	feature	lúc đặt
lead_time	int	feature	lúc đặt
arrival_date_year	int	not_used	lúc đặt
arrival_date_month	category	feature	lúc đặt
arrival_date_week_number	int	not_used	lúc đặt
arrival_date_day_of_month	int	not_used	lúc đặt
arrival_date	date	split_key	lúc đặt
stays_in_weekend_nights	int	feature	lúc đặt (có thể đổi sau)
stays_in_week_nights	int	feature	lúc đặt (có thể đổi sau)
adults	int	feature	lúc đặt (có thể đổi sau)
children	int	feature	lúc đặt (có thể đổi sau)
babies	int	feature	lúc đặt (có thể đổi sau)
meal	category	feature	lúc đặt
country	category (ISO 3166-1 alpha-3)	excluded_suspect	không rõ
market_segment	category	feature	lúc đặt
distribution_channel	category	feature	lúc đặt
is_repeated_guest	int (0/1)	feature	lúc đặt
previous_cancellations	int	feature	lúc đặt
previous_bookings_not_canceled	int	feature	lúc đặt
reserved_room_type	category (mã ẩn danh)	feature	lúc đặt
assigned_room_type	category	excluded_leakage	sau đặt
booking_changes	int	excluded_leakage	sau đặt
deposit_type	category	sensitivity_only	không rõ
agent	category (ID)	excluded_suspect	không rõ
company	category (ID)	excluded_suspect	không rõ
days_in_waiting_list	int	excluded_leakage	sau đặt
customer_type	category	feature	lúc đặt
adr	float	sensitivity_only	không rõ
required_car_parking_spaces	int	excluded_suspect	sau đặt (nghi lúc check-in)
total_of_special_requests	int	sensitivity_only	không rõ
reservation_status	category	excluded_leakage	sau đặt
reservation_status_date	date	excluded_leakage	sau đặt

Phụ lục D. Chuẩn bị vấn đáp

(a) Thời điểm dự đoán quyết định tập đặc trưng thế nào?

Mô hình chạy khi booking vừa được tạo, nên chỉ được dùng thông tin đã có và không bị cập nhật sau thời điểm đó. Mọi biến phát sinh sau (trạng thái đặt phòng, phòng được xếp, số lần sửa booking, số ngày chờ) hoặc nghi được ghi sau (chỗ đậu xe, tiền cọc, giá, yêu cầu đặc biệt) đều bị loại, dù chúng làm điểm số tăng (ablation A2, A3). Thời điểm dự đoán còn quyết định cách chia dữ liệu: chia theo ngày đến để mô phỏng dự đoán tương lai từ quá khứ.

(b) Log loss đo gì khác accuracy?

Accuracy chỉ đếm dự đoán đúng sau khi cắt ở một ngưỡng và bỏ qua độ tự tin. Log loss = −trung bình log xác suất gán cho kết quả thật: phạt rất nặng dự đoán tự tin mà sai, nên đo chất lượng của chính con số xác suất. Với dữ liệu mất cân bằng, mô hình luôn đoán "không hủy" vẫn có accuracy 63% nhưng log loss rất xấu. Nhóm chọn tham số theo log loss vì sản phẩm trả về xác suất.

(c) Odds ratio có phải tác động nhân quả không?

Không. Odds ratio = exp(hệ số) mô tả odds hủy thay đổi bao nhiêu lần khi một biến thay đổi và các biến khác trong mô hình giữ nguyên. Đó là liên hệ có điều kiện trong dữ liệu quan sát, có thể do biến gây nhiễu, tương quan giữa các biến hoặc cách chọn tham chiếu. Ví dụ odds ratio 72 của "đã hủy 1 lần" phản ánh booking đoàn bị hủy hàng loạt, không phải việc hủy trước đó gây ra việc hủy lần này. Muốn biết nhân quả cần thí nghiệm có kiểm soát.

Phụ lục E. Kịch bản demo (5–7 phút)

1. (0:00–0:45) Chạy `make app`, mở http://localhost:8000: trang Giới thiệu (câu hỏi, phạm vi, cách dùng có trách nhiệm); rê chuột vào một biến đã loại để xem lý do.
2. (0:45–2:30) Trang Dự đoán: "Điền booking mẫu" → "Rủi ro thấp" → Dự đoán (nhóm thấp); rồi "Rủi ro cao" (nhóm rất cao). Chỉ thanh ba ngưỡng, mở "Vì sao ra kết quả này?" để giải thích đóng góp log-odds.
3. (2:30–3:30) Input xấu: gõ chữ vào ô lead time → "Phải là số nguyên"; xóa trống → "Thiếu trường bắt buộc"; nhập 900 → nhãn "ngoài miền", vẫn gửi được và hiện cảnh báo từ API. Mở "Request/Response JSON", bấm Sao chép.
4. (3:30–4:30) Nhấn Ctrl/⌘ K, gõ không dấu "thoi gian dat truoc" → nhảy tới dòng lead_time trong bảng odds ratio; nhấn mạnh chú thích "không phải tác động nhân quả". Mở /docs, gửi thêm trường country để thấy lỗi 422.
5. (4:30–6:30) Đánh giá mô hình: Tổng quan (KPI so với baseline, bảng Validation/Test), Ngưỡng (ba ma trận nhầm lẫn, kéo thanh trượt khám phá), Calibration, Phân tích lỗi (Resort, Online TA, theo tháng); kết thúc ở Model card.

Phụ lục F. Phân công, nhật ký và khai báo công cụ AI

Bảng phân công hai thành viên, tự đánh giá theo rubric và nhật ký 6 tuần nằm ở `docs/contribution.md` và `docs/worklog.md` (nhóm tự điền; Git history là minh chứng).

Khai báo công cụ AI: nhóm sử dụng Claude Code (Claude Opus 5.5, Anthropic) để rà soát repo so với tài liệu giao nhiệm vụ, viết lại pipeline dữ liệu/mô hình/đánh giá, ứng dụng web, test tự động và soạn bản nháp tài liệu, báo cáo. Cách kiểm chứng: chạy lại toàn bộ pipeline trên môi trường sạch; 51 test tự động và 17 kịch bản end-to-end; đối chiếu mọi con số trong báo cáo với `models/metrics.json` và `reports/*.csv` (các bảng trong báo cáo được sinh trực tiếp từ các tệp này); kiểm tra lại từng quyết định rò rỉ bằng dữ liệu. Nhóm chịu trách nhiệm toàn bộ nội dung.